

Thermal Monitor

Thermal Monitor is a small embedded-style temperature monitor implemented in C++ and C. It reads a temperature sensor, reads a sensor type configuration, and drives three status LEDs:

- Green: normal temperature
- Yellow: warning temperature
- Red: critical high/low temperature or sensor/config error blink

The project also contains a Qt simulator, unit tests, Linux hardware adapters, and a QEMU-based hardware integration test.

Architecture diagram: [architecture.svg](#)

Prerequisites

On Debian/Ubuntu, install the build dependencies:

```
sudo apt update
sudo apt install git ca-certificates build-essential cmake libgtest-dev qtbase5-dev qt5-qmake
```

The Qt simulator requires Qt development files. This project can use Qt 6 when available and falls back to Qt 5. On Ubuntu 24.04, qt5-default is no longer available; use qtbase5-dev and qt5-qmake, or install the Qt 6 development package instead.

For QEMU hardware integration tests, install the host tools:

```
sudo apt install qemu-system-x86 qemu-utils cloud-image-utils openssh-client rsync
```

The QEMU integration test requires an x86_64 QEMU binary. On Debian/Ubuntu, qemu-system-x86 provides qemu-system-x86_64.

Behavior

All temperatures are evaluated internally in deci-Celsius.

Range	LED
'< 5.0 C'	Red
'>= 5.0 C && < 85.0 C'	Green
'>= 85.0 C && < 105.0 C'	Yellow
'>= 105.0 C'	Red

Sensor type conversion:

- TypeA: raw value is Celsius, converted with raw * 10
- TypeB: raw value is already deci-Celsius

On sensor or config read failure, the monitor turns green/yellow off and blinks red.

Both implementations also support an optional stability configuration:

- Median filtering: read sampleCount values with sampleSpacing between reads and evaluate the median.
- Hysteresis: keep Yellow or Red active until the converted deci-Celsius temperature moves below/above the configured exit margin.

Defaults preserve the direct behavior: one sample, no sample spacing, no hysteresis.

Project Layout

<code>thermal_monitor.cpp/.h</code>	C++ monitor core
<code>interfaces.h</code>	C++ sensor type definition
<code>platform/</code>	C++ Linux abstraction and mocks
<code>tests/</code>	C++ GoogleTest tests
<code>docs/</code>	Architecture/design documentation
<code>qemu/</code>	Reproducible QEMU VM setup
<code>c_impl/thermal_monitor.c/.h</code>	C monitor core
<code>c_impl/interfaces.h</code>	C Linux-like OS interface
<code>c_impl/linux_adapters.c/.h</code>	C Linux hardware adapters
<code>c_impl/mock_linux_os_c_adapter.hpp</code>	Adapter from C interface to C++ mock OS
<code>c_impl/qt_adapter.cpp/.hpp</code>	C++ wrapper for using the C monitor in Qt
<code>c_impl/tests/</code>	C GoogleTest tests
<code>demo_real_hw.cpp</code>	C++ real-Linux/QEMU hardware demo/test
<code>c_impl/demo_real_hw.c.c</code>	C real-Linux/QEMU hardware demo/test
<code>tools/gpio_sysfs_mock/</code>	Small kernel module for QEMU GPIO sysfs test
<code>scripts/run_qemu_hw_integration.sh</code>	Full QEMU hardware integration test runner

Architecture

C++ Path

ThermalMonitorCore contains the C++ monitor logic and depends on ILinuxLikeOs instead of calling Linux syscalls directly. This allows the same core to run against:

- MockLinuxOs in unit tests
- RealLinuxOs in the real-Linux/QEMU demo

The C++ monitor reads:

- `/dev/i2c-1` for the temperature sensor at address 0x48
- `/sys/bus/nvmem/devices/eeprom0/nvmem` for the sensor type
- `/sys/class/gpio/gpio10/value`
- `/sys/class/gpio/gpio11/value`
- `/sys/class/gpio/gpio12/value`

For I2C, it first tries I2C_RDWR. If that fails, it falls back to SMBus word read, which is required by Linux i2c-stub.

Optional stability behavior is configured through ThermalStabilityConfig and ThermalMonitor::setStabilityConfig(...).

C Path

The C core in `c_impl/thermal_monitor.c` mirrors the C++ path closely. It uses a small C vtable, CLinuxLikeOs, with Linux-like operations:

- open
- read
- write
- ioctl
- lseek
- close

That keeps the monitor testable without adding separate C-only sensor/config/GPIO mock layers. The C unit tests and the Qt adapter reuse the existing C++ MockSoC through `c_impl/mock_linux_os_c_adapter.hpp`, so the C and C++ monitors exercise the same simulated `/dev/i2c-1`, `NVMEM`, and `GPIO` files.

The real adapter in `c_impl/linux_adapters.c` maps `CLinuxLikeOs` to `POSIX/Linux` syscalls.

Optional stability behavior is configured through `ThermalStabilityConfig` and `thermal_monitor_set_stability_config(...)`.

C libraries are split by responsibility:

- `ThermalMonitorC`: C core plus the C Linux-like interface
- `ThermalMonitorCLinux`: Linux hardware adapters

Build Locally

```
cmake -S . -B build
cmake --build build
```

The default build includes:

- `ThermalSimulator Qt GUI`
- C++ core library
- C core library
- C Linux adapters
- C++ hardware demo
- C hardware demo
- unit tests

You can also use plain make after configuring:

```
make -C build -j$(nproc)
```

Run Unit Tests

```
ctest --test-dir build --output-on-failure
```

Expected result:

```
100% tests passed, 0 tests failed out of 18
```

The unit tests cover:

- `TypeA` thresholds
- `TypeB` thresholds
- sensor/I2C error handling
- config/EEPROM error handling
- callback behavior
- start/stop behavior
- sensor type is read once at startup
- median filtering
- hysteresis after sensor-type conversion

Run Qt Simulator

```
./build/ThermalSimulator
```

The simulator uses mock sensor/config/GPIO backends. It does not require real hardware or QEMU.

QEMU Hardware Integration Test

The QEMU integration test requires an x86_64 QEMU binary. On Debian/Ubuntu, install `qemu-system-x86`, which provides `qemu-system-x86_64`.

The automated QEMU test is:

```
scripts/run_qemu_hw_integration.sh
```

It expects the VM assets in:

```
qemu/
```

The script can be configured with environment variables:

```
QEMU_DIR=/path/to/qemu-dir \  
VM_PORT=2222 \  
scripts/run_qemu_hw_integration.sh
```

Create the local VM image first:

```
qemu/create-vm-image.sh  
qemu/provision-vm.sh
```

The generated qcow2 image, cloud-init ISO, SSH key, and logs are ignored by Git. See `qemu/README.md`.

To return to a clean local QEMU state:

```
qemu/clean.sh
```

The script performs the complete flow:

1. Starts the QEMU Debian VM.
2. Waits for SSH.
3. Syncs this repository into the VM.
4. Builds and loads `tools/gpio_sysfs_mock/thermal_gpio_mock.ko`.
5. Exports GPIO 10, 11, and 12 through `/sys/class/gpio`.
6. Loads `i2c-dev` and `i2c-stub` `chip_addr=0x48`.
7. Builds the project in the VM.
8. Runs all unit tests.
9. Runs the C++ hardware demo/test.
10. Runs the C hardware demo/test.
11. Powers off the VM.

The integration test uses real Linux kernel interfaces for:

- I2C: `/dev/i2c-1` via `i2c-stub`
- GPIO: `/sys/class/gpio/gpio10..12/value` via the test GPIO kernel module

NVMEM is intentionally simulated with:

```
/tmp/demo_eeprom.nvmem
```

Expected final output includes:

```
cctest: 18/18 passed  
C++ demo_real_hw: Done: PASS.  
C demo_real_hw_c: Done: PASS.  
VM is down.
```

Manual Hardware Demo Setup

Inside a Linux VM or target with the required kernel interfaces:

```
sudo modprobe i2c-dev
sudo modprobe i2c-stub chip_addr=0x48
```

For QEMU GPIO sysfs testing:

```
cd tools/gpio_sysfs_mock
make
sudo insmod thermal_gpio_mock.ko
for n in 10 11 12; do
    [ -d /sys/class/gpio/gpio$n ] || echo $n | sudo tee /sys/class/gpio/export
done
```

Then run:

```
sudo ./build/demo_real_hw
sudo ./build/c_impl/thermal_monitor_c_demo_real
```

Notes

- GPIO sysfs is deprecated in modern Linux, but it is used here because the monitor requirements and existing code target `/sys/class/gpio`.
- The C core stays hardware-neutral; platform-specific behavior belongs in adapters.
- The C++ core stays syscall-neutral through ILinuxLikeOs.

Production Release Limitations

This project is an embedded-style demo and test exercise, not a complete production release. A production version should address these points:

- Add an external watchdog or supervisor that detects if the thermal monitor process or worker thread crashes and restarts or escalates safely.
- Add detailed structured error logging for I2C, NVMEM, GPIO, startup, shutdown, and callback failures.
- Extend the tests with real hardware-failure coverage, for example sensor disconnects, stuck bus behavior, GPIO export/write failures, corrupted NVMEM data, and permission errors.
- Replace fixed constants and magic numbers such as device paths, I2C address, GPIO numbers, thresholds, and timing values with validated configuration files or deployment configuration.
- Define behavior for live hardware swap or hotplug events. The current implementation assumes the sensor, NVMEM source, and GPIO interface remain stable while the monitor is running.